NafaPilot

Profit-aware merchant growth decision engine

Pitch, judge Q&A and improvement guide

What we are building, how to explain it, the questions judges are likely to ask with prepared answers, and ideas to make it stronger.

Team PayMatrix · Paytm Build for India AI Hackathon 2026 · Track 1: Merchant Growth AI

How to use this guide. Read chapters 1–4 to understand and tell the story. Use chapter 5 as a demo script. Rehearse chapter 6 (questions and answers) as a team; chapter 7 lists the weak spots to be honest about. Chapter 8 is a prioritised list of improvements: none of them are built yet.

All merchants, orders and outcomes in NafaPilot are synthetic, and campaign execution is simulated. It is an independent hackathon submission, not an official Paytm product, with no access to Paytm systems. Numbers quoted from demo runs come from that synthetic data. Status as of 29 October 2026 (Milestones 1–9 built).

Contents

1. The one-minute story
2. The problem in depth
3. What NafaPilot does, step by step
4. What makes it different
5. A 5-minute demo script
6. Likely judge questions, with answers
7. Weak spots: how to answer honestly
8. Ideas to improve it

1. The one-minute story

Problem. Small merchants run discounts to bring in more orders, but many discounts lose money. Part of the discount goes to customers who would have bought anyway, and nobody measures whether the campaign actually caused the extra orders.

Solution. NafaPilot is a decision engine for merchant campaigns. It spots a change in the merchant's sales, lets AI suggest campaign ideas, and then uses exact, tested code to calculate the *incremental profit* of each idea in a bad, expected and good case. The merchant's own limits (discount cap, maximum loss, budget) can veto any idea. The merchant approves the exact numbers. The campaign is then run as an experiment with a holdout group, so we measure what the campaign really caused, with an honest margin of error.

One line. *Gemini proposes; Python calculates; policy enforces; the merchant authorises; the experiment measures.*

The example we use everywhere: Anand Café (synthetic)

1. **Detect.** Weekday 2–5 pm orders fell from 38 to 30 a day (−21%) over 8 weeks. NafaPilot flags it and says the cause is unknown: order data alone can't say why.
2. **Propose.** The AI suggests (A) a ₹20 targeted offer to existing customers and (B) a ₹30 off-peak offer for everyone.
3. **Calculate.** A earns +₹100 / +₹2,400 / +₹4,700 in the conservative / expected / optimistic cases. B earns −₹3,600 / +₹1,200 / +₹5,400.
4. **Enforce.** B's expected profit looks fine, but its bad case loses ₹3,600, more than the merchant's ₹1,000 loss limit. B is blocked. A is recommended.
5. **Approve and simulate.** The merchant approves A; 1,109 eligible customers are split 887 (offer) / 222 (holdout).
6. **Measure.** One simulated run gave +106 extra orders (95% interval +32 to +181): a **positive** effect. Its measured profit was +₹6,308, but the interval (−₹404 to +₹13,019) crosses zero, so the profit verdict is **inconclusive**. The system doesn't call it a win.

2. The problem in depth

2.1 "More orders" is the wrong goal

A discount campaign has three effects:

- **Incremental orders (I):** new orders caused by the offer. Good: they bring margin.
- **Existing discounted orders (S):** customers who would have bought anyway now pay less. Pure cost.
- **Campaign cost (C):** messages, printing, setup.

A dashboard that shows "orders went up during the campaign" mixes I with ordinary orders and hides S. A campaign can increase orders and still lose money. NafaPilot's target is **incremental contribution profit**: $\Delta P = I \times (AOV - d - VC) - S \times d - C$.

2.2 Merchants can't tell cause from coincidence

Sales go up and down every week anyway. Without a comparison group, any rise after a campaign could be luck, a festival or the weather. The standard fix is a **holdout**: randomly keep some customers out of the offer and compare. Big companies do this routinely; small merchants never do.

2.3 AI alone is risky for money decisions

A language model can write a convincing campaign plan, but it can also invent numbers, get arithmetic wrong, or confidently explain a sales drop with a cause it made up. For a merchant's money, every figure must be reproducible and every rule must hold whatever the AI says.

2.4 Real data is messy

Orders are missing costs, many customers are anonymous walk-ins, and some days have no records (closed? outage? no sales?). A trustworthy tool must say "insufficient data" instead of guessing, and must not treat missing as zero.

3. What NafaPilot does, step by step

Step	What happens	Who / what does it
1 Detect	Compares the last 2 weeks with a reference period for each time slot. Flags changes that pass a statistical test (controlling false alarms across slots) *and* are at least 10%. Shows a 4-week "if nothing changes" projection, clearly labelled.	Analytics engine (M4)
2 Propose	Sends observed facts, the merchant's settings and limits, preferences and past decisions to the AI. It returns 1–5 structured candidates (the mock returns three, including "no action"): type, discount, target group, assumed I and S for three scenarios, and a rationale.	Gemini or the offline mock (M2, M7)
3 Calculate	Exact ₹ profit for each scenario, break-even orders, and the discount handed out. Missing inputs → "insufficient data", never a guess.	Profit engine (M5)
4 Enforce	Six rules: valid inputs, discount cap, margin after discount, worst-case loss, campaign budget, merchant approval.	Policy engine (M5)
5 Select & explain	A fixed rule picks the best permitted option (highest expected profit, then safest). The AI writes an explanation in English or Hinglish; it is kept only if every number in it matches the calculations.	Code rule + grounded AI (M7)
6 Approve	The merchant approves or rejects one exact version. Any edit creates a new version that needs a new approval.	Merchant (M6)
7 Simulate	The approved version runs once in simulation: eligible customers split 80/20, outcomes simulated from the synthetic data.	Experiment engine (M8)
8 Measure	Treatment vs holdout difference with a 95% interval; measured profit with its interval; verdicts that stay "inconclusive" when the data can't decide.	Experiment engine (M8)
9 Learn	When rejecting, the merchant can set an explicit preference (e.g. "max discount ₹20"). It is applied to the next run as a filter in code. The model is not retrained.	Feedback (M7)

Everything is recorded in an append-only audit trail and shown in the merchant dashboard (M9).

4. What makes it different

Typical AI growth tool	NafaPilot
Optimises orders or revenue	Optimises incremental **profit**, including the cost of discounting orders that would have happened anyway
One predicted number	Three scenarios plus break-even; the risk limit is checked on the **bad** case
AI decides and explains	AI only suggests and words things; code decides; numbers in AI text are checked
"Sales rose 20% after the campaign"	Holdout comparison with a confidence interval; "inconclusive" is a valid answer
Guesses causes ("competitor opened")	Cause is always "unknown" unless data shows it
Fills in missing data	Reports "insufficient data" and what is missing
Approval is a button	Approval is bound to a fingerprint of the exact inputs; edits void it; the database enforces it
Learns by retraining on everything	Learns only from explicit merchant preferences; free text never silently becomes a rule

5. A 5-minute demo script

Time	Do	Say
0:00	Home page, pick Anand Café	"A café's weekday-afternoon orders fell 21%. Everything here is synthetic data, and execution is simulated."
0:30	Point at the opportunity card and projection	"We flag the change and say the cause is unknown. The projection is labelled as a projection, not a result."
1:00	Enter costs, click Get recommendations	"The AI doesn't know costs, so the merchant enters them. The AI proposes ideas; it doesn't do maths."
1:30	Recommendation page: A vs B	"Python calculates profit in three cases. B looks good on average, but its bad case loses ₹3,600, over the ₹1,000 limit, so policy blocks it. A is recommended by a fixed rule."
2:15	Explanation, switch to Hinglish (Explain again)	"The explanation is checked: any number not in our calculations and the template is used instead."
2:45	Open option B campaign page	"Approve is disabled. Approval can never override policy."
3:00	Open option A, Approve	"Approval is tied to these exact numbers. Approving doesn't run anything."
3:20	Run simulation… → confirmation → Confirm	"This is a simulated run with a 20% holdout. Nothing is sent to anyone."
3:50	Experiment page: verdicts, interval chart	"Prediction, observed outcomes, the estimated effect and measured profit are kept separate. If the interval crosses zero, we say inconclusive, even if the number looks good."
4:30	Caveats, audit trail	"Every step is recorded and can't be edited. Here's what the simulation can and can't tell you."

> **Before the demo:** generate the dataset, run the migrations, start both servers, use `LLM_PROVIDER=mock` unless Gemini is tested and the Wi-Fi is reliable, and do one full rehearsal on the demo machine. Each campaign version runs once, so create a fresh recommendation for each rehearsal. Because outcomes are random per version, the verdict can differ between runs; that is expected (see Q31).

6. Likely judge questions, with answers

A. Problem and product

Q1. Who is the user?
A small offline merchant (café, kirana, salon, pharmacy) who already takes digital payments and wants more sales without losing money. The dashboard is for the merchant; the engine could also serve a field sales or merchant-success team.

Q2. Why not just give a discount? Merchants know their business.
They know their customers, but not how much of a discount goes to people who'd have bought anyway. In our example, option B's expected case is positive, yet the bad case loses ₹3,600. Showing the range and enforcing their own loss limit protects them from a campaign that "worked" but cost money.

Q3. What's the business value for a payments company?
Healthier merchants transact more and stay longer. A tool that proves which campaigns pay off builds trust, and the decision layer could sit on top of existing offer and messaging tools. We have no access to Paytm systems; this is a design proposal.

Q4. How would it connect to real merchant data?
The engines take plain tables: orders (time, value, optional cost and customer ID), merchant settings and policy. The data layer (`DataRepository`) is the only part that reads the CSVs; replacing it with a connector to real transaction data, with merchant consent, leaves the engines unchanged.

Q5. What does the merchant actually do?
Look at the flagged change, enter campaign costs, review the options, approve or reject (optionally setting a preference), run the simulation, read the result.

B. The AI

Q6. If code does the maths, why use an LLM at all?
For what LLMs are good at: turning facts and constraints into campaign ideas, and explaining results in plain English or Hinglish, including answering "why not the off-peak offer?" We keep them away from what they're bad at: arithmetic and decisions.

Q7. What if Gemini hallucinates?
Three guards. Its output must match a strict schema or it is rejected. Its numbers (assumed orders) go through the profit and policy engines, which can block anything. Its explanations are kept only if every number appears in our calculations; otherwise a fixed template is used.

Q8. What if Gemini is down or out of quota?
Errors are typed (timeout, quota, invalid response…) and returned clearly; nothing is half-written. The offline mock provider runs the whole app with no key, which is also what all automated tests use.

Q9. Why not fine-tune a model on merchant data?
No real data to train on, and it isn't needed: the decisions are made by transparent formulas and rules. "Learning" here means explicit preferences and decision history, and every run states `model_retrained: false` .

Q10. Could a prompt injection make it approve a bad campaign?
No. The AI can't approve, can't change policy and can't set profit. Even a candidate whose rationale claims "approved and safe" is still evaluated and blocked by policy; there's a test for exactly that.

Q11. Which model and what does it cost?
Configurable; currently `gemini-3.8-flash` . One recommendation is one candidate call plus one explanation call, so cost per merchant is small. The key stays on the backend only.

C. The profit maths

Q12. Where do I and S (the assumed orders) come from?
Today, from the AI's suggestion or the merchant's edit, always labelled with their source. They are assumptions, which is why there are three scenarios and why the experiment exists. The natural improvement is to calibrate them from past experiments (chapter 8).

Q13. Why check the loss limit on the conservative case?
The merchant's "maximum loss" is about what happens if things go badly. Checking it on the average would let a risky campaign through.

Q14. Aren't you double-counting the discount?
No. The discount is counted once, inside net revenue: incremental orders at the discounted price, plus the discount lost on existing orders. The total discount given is shown separately for transparency but never subtracted again.

Q15. What's break-even?
The number of extra orders needed to cover the discount on existing orders plus the campaign cost. Campaign A needs 26; its expected case is 60.

Q16. Why Decimal instead of floats?
Floating point can't store ₹0.10 exactly. Decimal gives exact paise, so every figure can be checked by hand.

Q17. What does the formula not model?
Orders pulled forward from later weeks, changes in basket size, and effects on other products. These are inside the I and S assumptions, and the experiment measures the real net effect.

D. Statistics and experiments

Q18. Why a holdout?
It's the only way to separate what the campaign caused from what would have happened anyway. Holdout customers get nothing different, so the difference between groups is the campaign's effect.

Q19. Why 80/20?
It's the merchant's policy setting (`holdout_pct`). 20% keeps most customers in the offer while still giving a comparison group. A 50/50 split would measure more precisely but deny the offer to more people.

Q20. How is the split done? Could it be biased?
By a SHA-256 hash of the customer ID and an experiment seed: effectively random, reproducible, and independent of customer attributes. The same version always produces the same split.

Q21. Why Welch's t-interval?
It's the standard interval for a difference in means and doesn't assume both groups have the same spread. We use SciPy's implementation rather than our own. It's approximate for skewed counts, so we require at least 30 measured customers per group before giving a verdict.

Q22. What does "inconclusive" mean?
The 95% interval includes zero, or there's too little data. The campaign might have worked, or not; the data can't tell. We never turn a positive point estimate into a "win".

Q23. Effect positive but profit inconclusive. How?
The extra orders are clearly above zero, but the profit interval is wider, because each extra order is worth ₹70 while discounts on existing orders and campaign cost are subtracted. A real but small effect can still be uncertain in money terms.

Q24. Do you correct for multiple testing?
In detection (M4), yes: Benjamini–Hochberg across time slots. In experiments, one pre-specified metric is tested once per run, so no correction is needed; looking at many metrics or peeking repeatedly would need one, and we say so.

Q25. Is withholding an offer from 20% of customers fair?
They aren't harmed; they get the normal experience. It's standard practice, and it's the merchant's policy setting. Only by measuring can the merchant avoid wasting discounts on everyone.

Q26. Isn't the simulation circular, since its truth comes from your own prediction?
By default, yes, and we say so on screen: "agreement with the prediction is by construction; this tests the measurement pipeline, not the prediction." You can set a different true effect (including zero or negative) to see the system correctly report inconclusive or negative.

E. Data

Q27. Why synthetic data? How realistic is it?
No real merchant data is available, and using it would raise privacy issues. The generator plants known scenarios (a 21% decline, missing costs, a closure gap, a zero-effect campaign), so tests can prove the engines find the truth. It's simpler than reality (e.g. orders are Poisson), and we state that.

Q28. What happens with missing costs or anonymous customers?
Missing cost → profit "insufficient data" (merchant M007). Anonymous orders can't be assigned to a holdout, so they're counted and reported as excluded, never given invented IDs.

Q29. What about privacy?
The prototype has no personal data. A real deployment would use customer IDs only as opaque tokens, need merchant and customer consent under India's Digital Personal Data Protection Act, and keep data within agreed use.

F. Engineering

Q30. What stops a campaign running twice (double click, network retry)?
An idempotency key per request, a row lock, and database uniqueness on the version and the key. A retry with the same key returns the original result. Tested with 4 parallel requests on PostgreSQL: exactly one run.

Q31. Why did my second rehearsal give a different verdict?
Each new campaign version has its own random seed, so simulated outcomes differ; the same version always gives the same result. That variation is exactly why the confidence interval matters.

Q32. How is the audit trail protected?
Append-only in three layers: no update code, an ORM guard, and database triggers that reject UPDATE and DELETE.

Q33. Does it scale?
The engines are pure functions over one merchant's data, so merchants can be processed in parallel. PostgreSQL handles the writes. For many merchants, the next steps would be background jobs for detection and caching of baselines.

Q34. How is it tested?
242 backend tests (plus PostgreSQL variants) and 25 frontend tests, strict typing (mypy, TypeScript) and linting. Tests use the offline mock and never call Gemini.

Q35. Where are secrets?
Only in the backend's environment. The frontend never sees the Gemini key; the API reports only whether one is set; logs are redacted.

G. Tough questions

Q36. What's actually new here?
The combination: profit (not orders) as the objective, cannibalisation built in, risk checked on the bad case, AI strictly limited to suggestions and checked explanations, approvals bound to exact inputs, and every decision closed with a holdout measurement that is allowed to say "we don't know".

Q37. With 222 holdout customers, can you even detect the effect?
Honestly, often not. For Anand Café, the smallest effect we'd reliably detect is roughly 107 extra orders (80% power, approximate), while the expected case assumes 60. So most runs of campaign A would be inconclusive. That's a real finding, and the obvious next feature is a power check before running (chapter 8).

Q38. What if the merchant ignores the recommendation?
They can reject (optionally setting a preference) or edit. The edit is re-evaluated and still has to pass policy.

Q39. Why should a merchant trust a "no action" recommendation?
Because it's shown with the numbers: if no option has positive expected profit, spending money on a discount would lose it.

Q40. Is this an official Paytm product?
No. It's an independent hackathon prototype using synthetic data, with no access to Paytm systems.

7. Weak spots: how to answer honestly

Weak spot	Honest answer
Synthetic data only	"It demonstrates the method and that the engines find planted truths; it is not evidence about real merchants."
I and S are assumptions	"That's why we show three scenarios, check risk on the bad case, and measure with a holdout."
Low statistical power for small merchants	"Most single campaigns at one café will be inconclusive. We show that rather than hide it; pooling across merchants or longer runs would help."
Simulation truth defaults to the prediction	"Labelled on screen. Override the true effect to test other cases."
Welch is approximate for count data	"Standard and reasonable with ≥30 per group; we withhold verdicts below that."
Profit interval treats redemptions as fixed	"Stated in the results; the true uncertainty is slightly larger."
Alerts use nominal p-values	"It's a screening aid with false-alarm control, not a calibrated model; real orders vary more than our test assumes."
No deployment yet	"Milestone 10 covers deployment, end-to-end tests and an offline fallback."

8. Ideas to improve it

None of these are built. They're ordered by value for the hackathon versus effort.

8.1 Before the demo (high value, low effort)

Idea	Why it helps
Power check before "Run simulation"	Show "With 887/222 customers you can reliably detect about N extra orders; your expected case is 60." It turns the biggest weakness (Q37) into a feature: the tool warns before wasting a campaign. Pure maths on data we already have.
One-click demo reset / seeded demo campaigns	A script that creates fresh recommendation runs and one already-simulated example, so a rehearsal or live demo never hits "already executed".
"Predicted vs measured" summary card	One sentence on the experiment page: "Predicted +₹2,400 (range +₹100 to +₹4,700); measured +₹6,308 (−₹404 to +₹13,019): inconclusive." Judges remember one line.
List recommendation runs per merchant	A small read endpoint plus a list on the dashboard; removes the audit-trail workaround.
Deploy (M10)	A public link (Render + Vercel + Neon) and an offline fallback (mock LLM, local database) for the venue.

8.2 After the hackathon (medium effort)

Idea	Why it helps
Close the learning loop	Use measured results from past experiments to suggest I and S for new campaigns of the same type ("similar campaigns measured +0.07 orders per customer"). This is the "performance learning" the system currently marks as not yet available.
Variance reduction (CUPED)	Adjust outcomes using each customer's pre-campaign orders. Narrows intervals substantially, so smaller merchants get conclusive answers.
Probability of profit	Alongside the interval, report "chance the campaign made money" (e.g. via bootstrap). Easier for merchants to read than an interval.
Pooling across similar merchants	Combine evidence from many cafés running the same campaign type to estimate effects that one café alone can't.
Message preview	Generate the customer-facing offer text (WhatsApp/SMS style) in English and Hindi, clearly marked "preview, not sent".
Better uncertainty in measured profit	Bootstrap the profit interval so redemption uncertainty is included.

8.3 Towards a real product (larger effort)

- Real transaction-data connectors with consent, opaque customer tokens and data-retention rules (DPDP Act compliance).
- Real execution through an approved messaging/offer channel, behind the same approval and idempotency gates.
- Sequential testing (valid early stopping) and guard-railed adaptive allocation once many campaigns run.
- Multi-user roles (merchant, field agent, reviewer) with authentication.
- Monitoring: alert when measured results drift far from predictions, to recalibrate assumptions.

> **Pick one for the pitch.** If there's time for only one improvement, build the **power check**. It is cheap, uses data we already have, and directly answers the hardest statistics question a judge can ask.